CMPE3209 Computer Organization — Proje Teslim Tutanağı

M1 teslimi: Öğrenci bilgileri, Adım 0 ve Adım 1 · 2 Kasım 2026 (Pazartesi)

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Haydar Ali Atmaca
Öğrenci numarası (SID)	5001230016
E-posta	alihaydaratmaca58@gmail.com
Seçilen görev (G1–G5)	G2 — Sınav notu dağılımı
Seçilen ISA ve sürümü	RV32IM
Derleyici ve sürümü	RISC-V (32-bits) gcc 16.1.0 (Compiler Explorer), -march=rv32im -mabi=ilp32 -O0
Simülatör / araçlar	Ripes (ripes.dk web sürümü), ISA RV32IM; Compiler Explorer (godbolt.org)
Git deposu adresi	https://github.com/alihatmaca/Computer-Organization

Adım 0 — Kişiye özel parametreler

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001230016 → son hane 6	6
D2	SID'in son iki hanesi	5001230016 → son iki hane 16	16
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+1+6 = 18	18
N	24 + (D2 mod 25)	24 + (16 mod 25) = 24 + 16	40 eleman
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 6 satırı	64 byte / 16 byte line / direct / 4 line / set yok
Write policy	d çift → write-back, d tek → write-through	d = 6 çift	write-back
k	(S mod 7) + 3	(18 mod 7) + 3 = 4 + 3	7
Interrupt türü	S çift → Timer, S tek → I/O	S = 18 çift	Timer
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 6	1.6 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T2 = 20 + 5 × (6 mod 4) = 20 + 5 × 2 = 30	T1 = 1 cycle; T2 = 30 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (18 mod 7) = 2 + 4	6 (varsayım)
Görev parametresi (G2)	P = 50 + (SID mod 10); geçme notu P ve üstü	P = 50 + (5001230016 mod 10) = 50 + 6	P = 56

Not: SID $2^{31} - 1$'i (2 147 483 647) aştığı için programdaki sabit son 9 hane ile yazıldı: const int SID = 1230016;. S, tam numaranın haneleri toplamı olarak alındı; yönerge S için kısaltılmış numarayı belirtmiyor. Kısaltılmış numarayla S = 13 olurdu (k = 9, I/O, Nc = 8).

Adres alanları

Direct mapping, 32 bit adres (RV32): s + w = 32 bit; s = 28 bit.

Alan	Bit sayısı	Hesap
Word (w)	4 bit	log2(16 byte line) = 4
Line (r)	2 bit	log2(4 line) = 2
Tag (s − r)	26 bit	s − r = 28 − 2 = 26

Alan	Bit sayısı	Hesap
Toplam	32 bit	26 + 2 + 4 = 32

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi

☐ G1 Biyomedikal sinyal alarmı
☒ G2 Sınav notu dağılımı
☐ G3 Mesaj şifreleme ve bütünlük
☐ G4 Sıralama ve medyan
☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı

The program makes 40 exam grades from my student number with an LCG (linear congruential generator) and stores them in the global array notlar[40]. Then it goes through the array three times: first it generates the grades, second it counts them into the five grade ranges and adds them up, and third it marks the grades that are below the mean. The function grup() takes a grade and returns the range number (0 to 4) with a chain of if statements. The pass mark is P = 50 + (SID % 10) = 56. The results (bucket[5], mean, below_count and pass_count) are kept in global variables, so I can read them from memory in the simulator. The marking step (| 0x100) is done last, because it makes the grade bigger than 255 and it would break the counting if it was done earlier.

1.3 C kaynak kodu (satır numaralı)

```
 1  /* Haydar Ali Atmaca, 5001230016, G2 */
 2  const int SID = 1230016;
 3  int notlar[40];
 4  int bucket[5];
 5  int mean = 0, below_count = 0, pass_count = 0;
 6
 7  int grup(int g) {
 8      if (g < 50) return 0;
 9      if (g < 65) return 1;
10      if (g < 75) return 2;
11      if (g < 85) return 3;
12      return 4;
13  }
14
15  int main(void) {
16      unsigned int x = SID;
17      int toplam = 0;
18      int P = 50 + (SID % 10);
19
20      for (int i = 0; i < 40; i++) {
21          x = 1664525u * x + 1013904223u;
22          notlar[i] = (x >> 16) % 101;
23      }
24
25      for (int i = 0; i < 40; i++) {
26          toplam += notlar[i];
27          bucket[grup(notlar[i])]++;
28          if (notlar[i] >= P)
29              pass_count++;
30      }
31      mean = toplam / 40;
32
33      for (int i = 0; i < 40; i++) {
34          if (notlar[i] < mean) {
35              notlar[i] = notlar[i] | 0x100;
36              below_count++;
37          }
38      }
39      return 0;
40  }
```

1.4 Zorunlu yapı eşlemesi

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	3; 20, 25, 33	40 elemanlı global notlar dizisi üç döngüyle gezilir: üretim, sayım ve toplama, işaretleme.

#	Zorunlu yapı	Satır no	Açıklama
Y2	Koşullu dallanma	8–11, 28, 34	grup() içindeki if zinciri, geçme notu kontrolü ve ortalama karşılaştırması.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	7 (tanım), 27 (çağrı)	grup(int g) notu alıp 0–4 aralık numarası döndürür; 2. geçişte çağrılır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	21, 22, 35	Satır 21 çarpma ve toplama (LCG), satır 22 shift (>> 16) ve mod, satır 35 OR (| 0x100).
Y5	Dizi elemanına store	22, 35 (ayrıca 27)	notlar[i] satır 22 ve 35'te yazılır; bucket[] elemanı satır 27'de artırılır.
Y6	const int SID ve hesapta kullanımı	2; 16, 18	SID, LCG'nin başlangıç değeridir (satır 16) ve geçme notu P = 50 + (SID % 10) hesabında kullanılır (satır 18).
Y7	Sonucun tutulduğu global değişken(ler)	4–5	bucket[5], mean, below_count ve pass_count global değişkenlerdir.

1.5 Program çıktıları

Program Ripes'te çalıştırılıp çıkış kodu 0 ile bittikten sonra Memory sekmesinden okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
bucket[0]	0x100000A4	16 (0x10)	m1_bellek.png
bucket[1]	0x100000A8	9 (0x09)	m1_bellek.png
bucket[2]	0x100000AC	3 (0x03)	m1_bellek.png
bucket[3]	0x100000B0	4 (0x04)	m1_bellek.png
bucket[4]	0x100000B4	8 (0x08)	m1_bellek.png
mean	0x100000B8	56 (0x38)	m1_bellek.png
below_count	0x100000BC	18 (0x12)	m1_bellek.png
pass_count	0x100000C0	22 (0x16)	m1_bellek.png

Kontrol: bucket toplamı 16 + 9 + 3 + 4 + 8 = 40 = N; below_count + pass_count = 18 + 22 = 40.

1.6 İzlenecek kernel

Alan	Değer
C satır aralığı	8, 13 ve 27 (grup() içinde if (g < 50) return 0 ve fonksiyon çıkışı; ardından bucket[grup(notlar[i])]++)
Başlangıç ve bitiş adresi (hex)	0x28 (bgt a4,a5,.L2) — 0x1ec (sw a4,0(a5))
Dinamik instruction sayısı	20 (la pseudo-instruction'ı auipc + addi olarak 2 sayıldı)
Hangi döngü iterasyonu	2. geçiş, i = 3 (notlar[3] = 19 → grup() 0 döndürür)

Dinamik instruction sırası (RISC-V gcc 16.1.0, -O0):

#	Instruction	Açıklama
1	bgt a4,a5,.L2	Koşullu branch, alınmaz (g = 19 ≤ 49)
2	li a5,0	Sabit yükleme (ALU)
3	j .L3	Koşulsuz atlama
4	mv a0,a5	Dönüş değerini a0'a koy
5	lw ra,28(sp)	Load
6	lw s0,24(sp)	Load
7	addi sp,sp,32	ALU

#	Instruction	Açıklama
8	jr ra	Return
9	mv a5,a0	Sonucu al
10–11	la a3,bucket	auipc + addi
12	slli a4,a5,2	ALU (shift)
13	add a4,a3,a4	ALU
14	lw a4,0(a4)	Load (bucket[0])
15	addi a4,a4,1	ALU
16–17	la a3,bucket	auipc + addi
18	slli a5,a5,2	ALU (shift)
19	add a5,a3,a5	ALU
20	sw a4,0(a5)	Store (dizi elemanı)

Kernel load, store, ALU, koşullu branch ve return içerir. Pencere bellekte ardışık değildir (j .L3 ve jr ra atlar); bu yüzden başlangıç ve bitiş adresi arasındaki fark 20 × 4 byte değildir. Programı Ripes'te çalıştırmak için GCC çıktısının başına 3 satırlık bir başlangıç şablonu (jal ra,main; li a7,10; ecall) eklendi; kernel bundan etkilenmez.

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____